PLaMo Paper Translator: Local, Sentence-Aligned Bilingual Reading of Academic PDFs

User Manual, Version 1.2 (Single-Column Edition)

The PLaMo Paper Translator Project

November 2026

Abstract

PLaMo Paper Translator is a desktop application that turns an academic PDF into a sentence-aligned bilingual document and keeps everything on the reader's own computer. The application extracts text, figures and numbered equations from the PDF, translates each sentence with PLaMo 3 Translate running locally through llama.cpp, and presents the result in a three-pane reader with a table of contents, cross-reference links, bookmarks, notes, pinned clips and a side panel for a Markdown summary of the paper. This manual is written in the form of a research paper so that it can double as a realistic test document: it contains numbered sections, figures, tables, display equations, an algorithm and a reference list. We describe installation (Section 3), the document pipeline (Section 4), translation (Section 5), the reading interface (Section 6), quality checks (Section 7) and the paper library (Section 8).

Keywords: machine translation, PDF processing, local inference, reading interface, PLaMo

1 Introduction

Reading papers in a second language is slow, and pasting text into an online translator breaks figures, equations and the structure of the document. Many papers are also confidential before publication, which rules out cloud services. PLaMo Paper Translator addresses both problems. All processing happens on the local machine, and the original structure of the paper is preserved: headings become a table of contents, figures and equations stay next to the text that refers to them, and every sentence is shown beside its translation.

Figure 1 shows the reader. The original text appears on the left, the translation in the middle and the reader's notes on the right. Each row is one segment, labelled with its paragraph and sentence number.

The main contributions of the application are as follows.

- A fully local pipeline that needs no network connection after the first setup.
- Structure-aware extraction that keeps headings, captions, numbered equations and reference entries intact.
- Context-aware translation that sends neighbouring sentences from the same paragraph to the model.
- A research-oriented reader with cross-reference jumps, notes, bookmarks, clips, a summary panel with mathematics and figure references, a quality check and a searchable library.



Figure 1: The reading interface. The outline (left) is generated from the section structure of the paper. Each row pairs one source sentence with its translation and a note field; the bookmark list is shown on the right.

2 System Overview

2.1 Architecture

The application consists of three processes, shown in Figure 2. A browser displays the reader UI. A small Python HTTP server, `app.py`, listens on `127.0.0.1:7865`; it extracts documents, runs translation jobs and stores results. Translation itself is performed by `llama-server` from llama.cpp [3], which serves the PLaMo 3 Translate model [1] on `127.0.0.1:8041`. Both servers bind to the loopback interface only, so no other computer can reach them.

2.2 Data Storage

All user data is kept in plain files under a single folder, summarised in Table 1. Deleting the folder removes every trace of a paper.

2.3 Hardware Requirements

The reference machine has an NVIDIA RTX 2000 Ada laptop GPU with 16 GB of video memory. Table 2 and Figure 3 compare the available quantised models. The 8B model is the default; the 31B model does not fit in 16 GB.

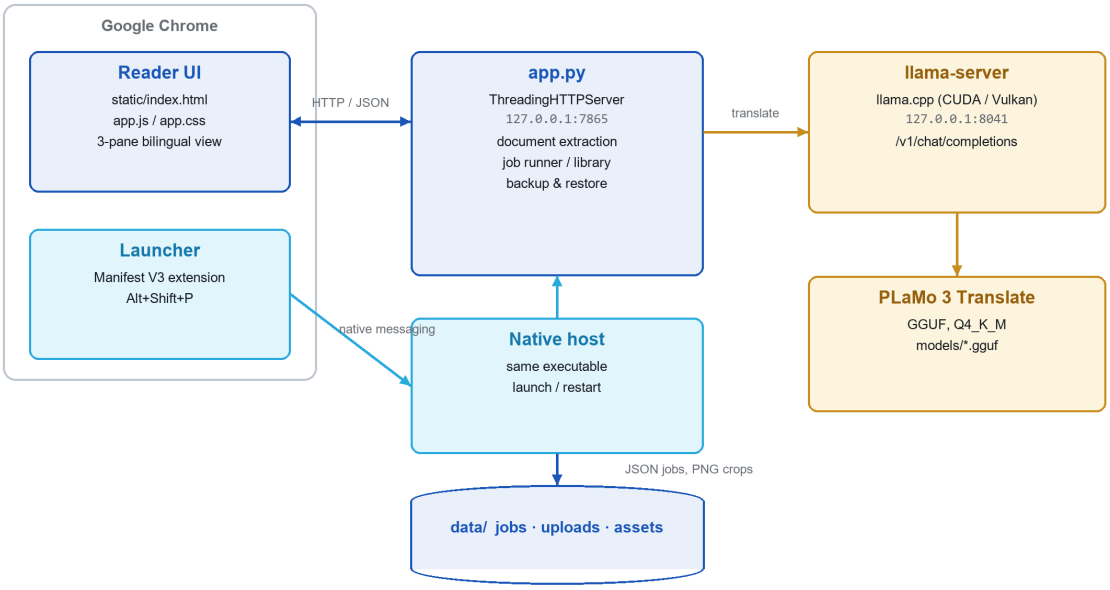


Figure 2: Architecture. The reader UI talks to `app.py` over HTTP; `app.py` calls llama-server through its OpenAI-compatible endpoint. The optional Chrome launcher starts or restarts the application through native messaging.

Table 1: Folders used by the application.

Folder	Contents
`data/uploads`	original PDF, TXT or Markdown
`data/jobs`	one JSON file per translation
`data/assets`	figure and equation crops (PNG)
`models`	GGUF model and chat template
`runtime`	llama.cpp binaries

Table 2: PLaMo 3 Translate models in Q4_K_M quantisation.

Model	Size (GB)	Bytes	Fits 16 GB
2B	1.81	1,807,203,040	yes
8B	5.28	5,280,445,728	yes (default)
31B	19.53	–	no

3 Installation

3.1 Setup Script

Double-click `Start-PLaMoPaperTranslator.cmd`. On the first run it shows the main points of the PLaMo Community License [2] and asks for confirmation. The setup script then performs five steps:

1. create a Python virtual environment and install pypdf [4], Pillow [6] and pypdfium2 [5];
2. download the CUDA build of llama.cpp, or the Vulkan build when no NVIDIA GPU is present;
3. download the model and its chat template from the official Hugging Face repository;

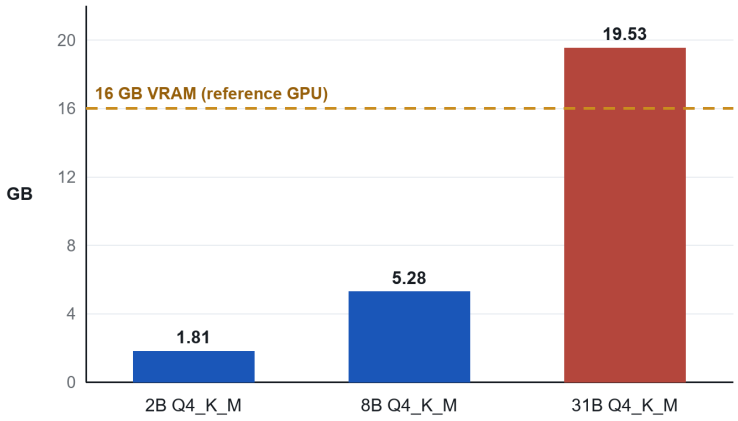


Figure 3: Model file sizes compared with the 16 GB video memory of the reference GPU.

4. verify the GGUF header, the file size and the SHA-256 digest;
5. start the application and open `http://127.0.0.1:7865`.

Interrupted downloads resume from a `.partial` file. A model file f is accepted only when

$$\text{size}(f) = b^\star \quad \wedge \quad \text{SHA256}(f) = h^\star, \tag{1}$$

where $b^\star$ and $h^\star$ are the values currently published by the official repository.

3.2 Chrome Launcher

The portable package includes an optional Chrome extension. After loading the extension and running `install-native-host.ps1`, the toolbar button or ALT+SHIFT+P restarts the application and opens the reader. If a translation is running, the application is not restarted, so no work is lost.

4 Document Pipeline

Figure 4 summarises how an uploaded file becomes a list of segments. Files of up to 80 MB in PDF, TXT or Markdown format are accepted.

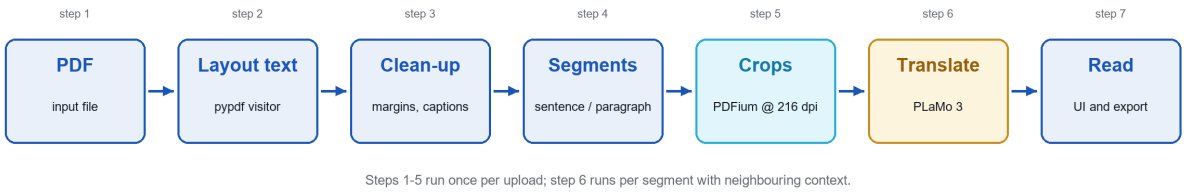


Figure 4: Document pipeline. Extraction and clean-up run once per upload; translation runs once per segment.

4.1 Text and Layout Extraction

Text is read with pypdf together with the position and font size of every text fragment. Running headers and footers are removed using these positions. Let $k(\cdot)$ map a small-font fragment near the top or bottom margin to a normalised key. The set of repeated margin keys is

$$R = \left\{ k \;\middle|\; |\{\, p : k \in K_p \,\}| \geq 2 \right\}, \tag{2}$$

where K_p is the set of keys found on page p. Fragments whose key is in R are dropped, as are page numbers.

4.2 Segmentation

The cleaned text is split into blocks and each block is classified as a heading, sentence, list item, caption, equation, table or citation. In sentence mode every sentence becomes a segment; in paragraph mode the sentences of one paragraph are kept together. A segment is identified by its paragraph number p and its position k in the paragraph,

$$s_{p,k}, \qquad p = 1, \ldots, P, \quad k = 1, \ldots, n_p. \tag{3}$$

Headings are assigned a level from their numbering, so "3.2" becomes level 2. Sentences broken by a page or column boundary are merged again, and each reference entry is merged into a single citation segment.

4.3 Figures and Equations

Figures are cropped from the rendered page rather than taken from the embedded images. Labels such as (a) and (b) that are drawn over a figure are therefore kept. A numbered display equation is detected from the position of its number at the right margin and rendered with PDFium at

$$\sigma = \frac{216 \text{ dpi}}{72 \text{ pt/in}} = 3 \tag{4}$$

pixels per point. The extracted text of the equation is kept for search and copying, but the image is treated as the authoritative form and is never translated.

4.4 Subscripts and Superscripts

Plain text extraction loses the difference between E_{algo} and "Ealgo". The application therefore compares each text fragment f with the fragment b before it. Let z denote the font size and $\Delta y = y_f - y_b$ the shift of the baseline. When $z_b \geq 1.15\, z_f$, the fragment is classified as

$$\text{role}(f) = \begin{cases} \text{superscript} & \text{if } \Delta y \geq 0.2\, z_b, \\ \text{subscript} & \text{if } \Delta y \leq -0.08\, z_b, \\ \text{normal} & \text{otherwise,} \end{cases} \tag{5}$$

so small capitals on the same baseline stay normal text. A script is stored in the text as `E_{algo}` or `b^{*}` and shown as a real subscript or superscript in the reader and in exports. Pieces of one script, such as p, a comma and k in $s_{p,k}$, are joined. In a test with 60 sentences, PLaMo 3 Translate kept all 126 marked scripts unchanged; a flattened script would be restored automatically.

5 Translation

5.1 Context Construction

PLaMo 3 Translate accepts optional context through its chat template [1]. For segment $s_{p,k}$ the application builds

$$c_{p,k} = \mathrm{trunc}_{1400}\big(u \,\|\, \mathrm{suf}_{600}(B_{p,k}) \,\|\, \mathrm{pre}_{600}(A_{p,k})\big), \tag{6}$$

where u is the reader's own context note, $B_{p,k} = s_{p,1} \cdots s_{p,k-1}$ and $A_{p,k} = s_{p,k+1} \cdots s_{p,n_p}$ are the sentences before and after it in the same paragraph, and $\|$ denotes concatenation. Figure 5 illustrates the construction.

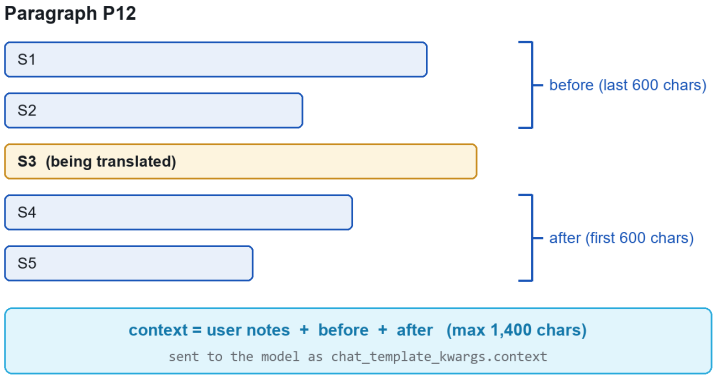


Figure 5: Context for one sentence. Only sentences from the same paragraph are used, so unrelated text never leaks into the prompt.

5.2 Request Parameters

Decoding is greedy (temperature 0). The output limit for a source segment of n characters is

$$T(n) = \min\big(2048,\ \max(384,\ \lfloor 1.35\,n \rfloor)\big) \tag{7}$$

tokens. Segments that contain almost no letters, such as a row of numbers, are copied unchanged. Algorithm 1 shows the complete loop. The job is written to disk every three segments, so at most three translations are lost if the computer stops.

5.3 Glossary

A glossary fixes the translation of important terms. Each line holds one source term and its required translation; the accepted separators are listed in Table 3. Lines starting with # are comments. In the table, T stands for the term in the target language.

5.4 Progress

While a job runs, the progress shown to the reader after m of N segments is

$$\pi = \frac{1}{10}\,\mathrm{round}\left(1000 \cdot \frac{m}{N}\right)\%, \tag{8}$$

that is, a percentage rounded to one decimal place.

Algorithm 1 Translation job

Require: segments $s_1, \ldots, s_N$, options θ
start llama-server if it is not healthy
for $i = 1$ **to** N **do**
  if the job was cancelled **then**
    return cancelled
  end if
  if s_i is an equation with an image **then**
    $t_i \leftarrow s_i$
  else
    $t_i \leftarrow \textsc{Translate}(s_i, c_i, \theta)$
  end if
  if $i \bmod 3 = 1$ **then**
    save job
  end if
end for
return completed

Table 3: Glossary syntax. All four lines define the same pair.

Line	Separator
`curved layer =` T	equals sign
`curved layer =>` T	ASCII arrow
`curved layer` $\rightarrow T$	Unicode arrow
`curved layer<TAB>`T	tab character

6 Reading Interface

6.1 Layout

The reader is not tied to the page size of the PDF. On a wide screen the outline is fixed on the left and the bookmark list on the right, as in Figure 1; while the summary panel is open, the bookmark list moves below the outline. On a tablet both lists move above the text, and on a narrow screen the note field moves below each pair.

6.2 Navigation and Cross-References

References such as “Fig. 3”, “Eq. (2)” and “Table 1” become links. When several segments carry the same number, the one with an image is preferred. Each jump is recorded, and the *Back* button returns to the previous position; the last 20 positions are kept. A figure that appears in the PDF far from its first mention is moved to just after that mention.

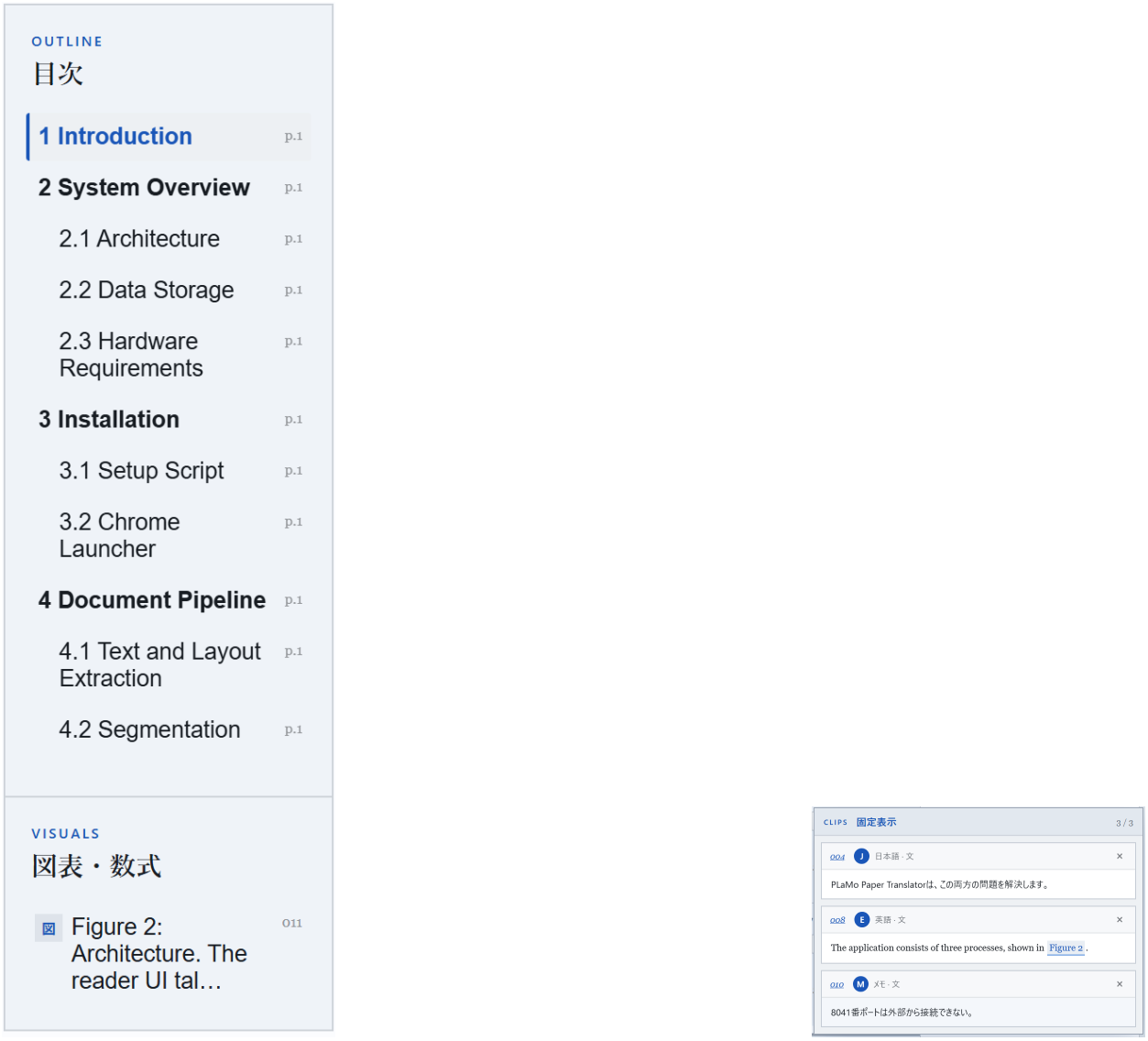
6.3 Notes, Bookmarks and Clips

Each segment has a note field that is saved automatically, and a star that adds it to the bookmark list. The source, translation or note of any segment can be *clipped*; clips stay visible in the lower-right corner while

reading. If $C_i \subseteq \{\text{source}, \text{translation}, \text{note}\}$ denotes the parts clipped from segment i, the application enforces

$$\sum_{i=1}^{N} |C_i| \leq 3. \tag{9}$$

Figure 6 shows the outline and the clip shelf.



(a) Outline **(b)** Clip shelf

Figure 6: Reading aids: (a) the outline generated from headings and (b) the clip shelf with pinned segments.

6.4 Paper Summary

The *Summary* button, or ALT+S, opens a panel at the right edge of the window (Figure 7). The page becomes narrower instead of being covered, so the paper can be read while the summary is written. The panel holds tags, the source URL, the reading status and free notes, and saves them automatically. Its width can be changed by dragging the left edge.

The notes are written in Markdown with the additions in Table 4. Mathematics uses TeX notation and is rendered with KaTeX [7], which is bundled with the application, so no network access is needed. The *To summary* button on every row inserts that row: a sentence becomes a quotation with a link back to the row, and a figure or equation becomes an embedded image. *Edit* and *View* switch between the text and the rendered notes; while editing, a paragraph that contains mathematics or references is previewed below the text.

Table 4: Additions to Markdown in the summary.

Notation	Result
`$\sigma = 216/72$`	inline mathematics
`$$ ... $$`	display mathematics
`Fig. 2, Eq. (4)`	link to the figure or equation
`![[Fig. 2]]`	embedded figure image
`[[#123]]`	link to row 123

6.5 Editing and Retranslation

The *Edit* button corrects a single translation. *Retranslate* sends one segment to the model again with the same context and glossary. The previous text is kept as a revision; up to 20 revisions per segment can be viewed and restored. Equation images cannot be edited.

6.6 Keyboard Shortcuts

Table 5 lists the shortcuts.

Table 5: Keyboard shortcuts.

Keys	Action
ALT+L	open the library
ALT+S	open or close the summary panel
ALT+SHIFT+P	restart and open (Chrome)
ESC	close the current dialog or panel

7 Quality Check

The quality check finds problems that can be detected mechanically. It does not replace proofreading by a person. Table 6 lists the checks. For a source s and translation t, a translation is reported as suspiciously short when

$$|s| \geq 80 \quad \wedge \quad |t| < \max\big(8,\ \lfloor 0.12\,|s| \rfloor\big), \tag{10}$$

and numbers are reported when the set difference

$$M = \mathcal{N}(s) \setminus \mathcal{N}(t) \tag{11}$$

is not empty, where $\mathcal{N}(\cdot)$ extracts numbers together with %, °C and °F.

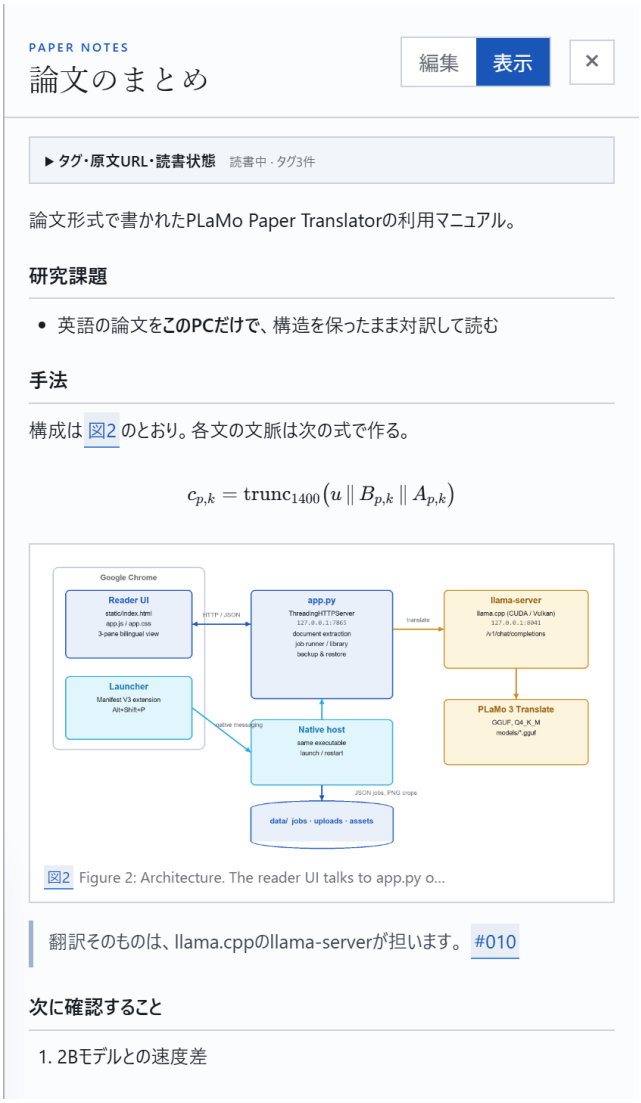


Figure 7: The summary panel showing rendered notes: headings, a display equation, an embedded figure and a quoted row with a link back to the text.

Table 6: Checks performed by the quality check.

Check	Level	Condition
Missing	error	no translation
Unchanged	warning	$t = s$, ≥ 3 words
Numbers	warning	Eq. (11)
Short	warning	Eq. (10)
Glossary	info	target term absent

8 Paper Library

The library lists every saved translation (Figure 8). It searches titles, tags, summaries, source text, translations and notes, and filters by reading status: unread, reading, completed or archived. Opening a paper resumes at the last segment read. For a paper with N segments whose last read segment has index r (counting from zero), the progress bar shows

$$\rho = \operatorname{round}\left(100 \cdot \frac{r+1}{N}\right)\%. \tag{12}$$

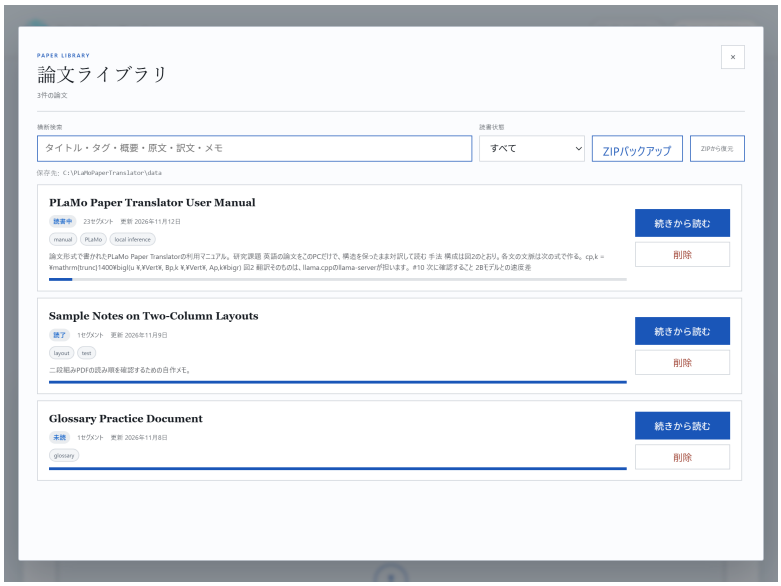


Figure 8: The paper library with search, reading-status filter, backup and restore.

8.1 Backup and Restore

ZIP backup saves jobs, original files and crops in one archive. *Restore from ZIP* only adds files that do not exist yet; it never overwrites. Before anything is written, the archive must satisfy

$$\text{upload size} \le 2\ \text{GB}, \tag{13}$$

$$\textstyle\sum \text{expanded size} \le 8\ \text{GB}, \tag{14}$$

$$\text{number of files} \le 100{,}000, \tag{15}$$

and every path must lie inside `data/jobs`, `data/uploads` or `data/assets`.

8.2 Deleting a Paper

Delete removes a translation after confirmation. The original file and crops are deleted as well, unless another translation was made from the same file. A paper that is being translated cannot be deleted.

9 Export and Licensing

Table 7 lists the export formats. The summary is placed at the top of the HTML and Markdown exports. In HTML, references link to the rows of the same file, and KaTeX is embedded only when the summary

contains mathematics. Every export carries the attribution “Built with PLaMo” and names the model, as required by the PLaMo Community License [2]. Use within an organisation can count as commercial use under that license; check its conditions before deploying the application at work.

Table 7: Export formats.

Format	Contents
HTML	single file with embedded figures, notes and the rendered summary
Markdown	summary, then source and translation, one pair per entry
JSON	all segments, tags, summary and reading status

When publishing translations of other people’s papers, check the license of each paper. A license that allows adaptation, such as CC BY 4.0 [9], is required, because a translation is an adapted work.

10 Limitations

The application needs a PDF that contains text; scanned pages must be processed with OCR first. Two-column layouts are read in the order stored in the PDF, which is usually but not always the reading order. Figures drawn only with vector graphics are not cropped, and their text is kept as ordinary segments. Inline mathematics is translated as text; subscripts and superscripts are kept (Section 4.4), but fractions and stacked limits inside a sentence become flat scripts. Finally, machine translation can be wrong: results used for submissions, contracts or medical decisions must be checked by an expert.

11 Conclusion

PLaMo Paper Translator brings structure-aware, sentence-level translation of academic papers to an ordinary laptop without sending any text to the cloud. We hope this manual is useful both as a guide and as a test document. Because it is dedicated to the public domain, it can be translated, shown in videos and redistributed without restriction.

Acknowledgments

The application is built with PLaMo. It relies on llama.cpp, pypdf, Pillow, PDFium and KaTeX, and we thank their developers.

References

[1] Preferred Networks, Inc. PLaMo 3 Translate 8B (GGUF). `https://huggingface.co/pfnet/plamo-3-translate-8b-GGUF`, 2026.

[2] Preferred Networks, Inc. PLaMo Community License. `https://plamo.preferredai.jp/info/plamo-community-license-ja`.

[3] G. Gerganov and contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`.

[4] pypdf contributors. pypdf: a pure-Python PDF library. `https://pypdf.readthedocs.io/`.

[5] pypdfium2 contributors. pypdfium2: Python bindings to PDFium. https://github.com/pypdfium2-team/pypdfium2.

[6] J. A. Clark and contributors. Pillow: the friendly PIL fork. https://python-pillow.org/.

[7] KaTeX contributors. KaTeX: the fastest math typesetting library for the web. https://katex.org/.

[8] Google. Native messaging. Chrome for Developers documentation, https://developer.chrome.com/docs/extensions/develop/concepts/native-messaging.

[9] Creative Commons. Attribution 4.0 International (CC BY 4.0). https://creativecommons.org/licenses/by/4.0/.

[10] Creative Commons. CC0 1.0 Universal Public Domain Dedication. https://creativecommons.org/publicdomain/zero/1.0/.

A HTTP Interface

Table 8 lists the main endpoints of `app.py`. They are intended for the bundled reader and the Chrome launcher [8]; they are not a stable public interface.

Table 8: Main HTTP endpoints.

Request	Purpose
`GET /api/status`	engine and model state
`POST /api/documents`	upload and extract a file
`POST /api/jobs`	start a translation
`GET /api/jobs/{id}`	job state and segments
`GET /api/jobs/{id}/export`	HTML, MD or JSON
`GET /api/jobs/{id}/quality`	quality check
`DELETE /api/jobs/{id}`	delete a paper
`GET /api/library`	search the library

B Troubleshooting

- **The engine does not start.** Check `data/llama-server.log`. Loading must finish within three minutes.
- **Nothing is extracted.** The PDF probably contains only images; run OCR first.
- **The Chrome button reports an error.** Run `install-native-host.ps1` again and restart Chrome.

This manual is dedicated to the public domain under CC0 1.0 [10]. It may be copied, translated and redistributed for any purpose without permission or attribution.